

```
In [1]: ClearAll["Global`*"];
$HistoryLength=0;
Print["====="];
Print["S124-T5R1 STRICT FRESH-WORLD"];
Print["S119B NEURALIZED GENERALIZATION ATTRIBUTION"];
Print["PAIRWISE TRAINING -> UNSEEN HIGH-ORDER COMPOSITIONS"];
Print["====="];
Print["WolframVersion=", $Version];
Print["Date=",DateString[]];
T5SStop[msg_] := (Print["FATAL: ", msg]; Abort[]);
publicActionsS119B = Range[0, 7];
publicProbesS119B = Range[0, 13];
supportScanDepthS119B = 8;
localBFSSafetyS119B = 128;
preparationBFSSafetyS119B = 256;
hiddenBenchmarkSeedS119B = 1245501;
t5sPerceptionSensorDim = 12;
t5sPerceptionPosDim = 8;
t5sPerceptionMicroLen = 4;
t5sPerceptionScale = 2.;
t5sPerceptionNoise = 0.30;
t5sPerceptionDrift = 0.12;
t5sPerceptionGainJitter = 0.12;
t5sPerceptionDModel = 64;
t5sPerceptionHeads = 4;
t5sPerceptionLayers = 2;
t5sPerceptionFF = 192;
t5sPerceptionDropout = 0.10;
t5sPerceptionLR = 0.0003;
t5sPerceptionBatch = 64;
t5sPerceptionRounds = 20;
t5sPerceptionPatience = 5;
t5sPerceptionTrainPerClass = 1200;
t5sPerceptionValPerClass = 300;
t5sPerceptionPrototypeSeed = 1245601;
t5sPerceptionTrainSeed = 1245602;
t5sPerceptionValSeed = 1245603;
t5sPerceptionNetSeed = 1245604;
t5sPerceptionQuerySeed = 1245605;
t5sReasonDModel = 64;
t5sReasonHeads = 4;
t5sReasonLayers = 2;
t5sReasonFF = 192;
t5sReasonDropout = 0.10;
t5sReasonPosDim = 8;
t5sReasonLR = 0.0003;
t5sReasonBatch = 64;
t5sReasonRounds = 35;
t5sReasonPatience = 6;
t5sReasonSplitSeed = 1245701;
t5sReasonSelectSeed = 1245702;
t5sReasonFinalSeed = 1245703;
t5sTargetDevice = "CPU";
t5sPerceptionGate = 0.98;
```

```

t5sTrainingPerceptionGate=0.98;
t5sReasonValidationBalancedGate=0.60;
t5sReasonTrainingBalancedGate=0.70;
t5sWinnerMargin=0.02;
If[OddQ[t5sPerceptionPosDim],T5SStop["perception positional dimension must be even"]
If[OddQ[t5sReasonPosDim],T5SStop["reason positional dimension must be even"]];
If[Mod[t5sPerceptionDModel,t5sPerceptionHeads]!=0,T5SStop["perception dModel/head
If[Mod[t5sReasonDModel,t5sReasonHeads]!=0,T5SStop["reason dModel/head mismatch"]];
t5sOutputDirectory=FileNameJoin[{Directory[],"S124_T5R1_Output"}];
If[!DirectoryQ[t5sOutputDirectory],CreateDirectory[t5sOutputDirectory]];
discoveryPolicySpecS119B=<|
"Actions"->publicActionsS119B,
"Probes"->publicProbesS119B,
"SupportScanDepth"->supportScanDepthS119B,
"FactorRule"->"EqualObservedProbeSupport",
"LocalLearning"->"BFSAtBaselineContext",
"InteractionDiscovery"->"OneCandidateContextFactorAtATime",
"TransitionRepresentation"->"SparseConditionalLocalTable",
"MaximumTrainingInteractionOrder"->2
|>;
t5sReasonSpec=<|
"DModel"->t5sReasonDModel,
"Heads"->t5sReasonHeads,
"Layers"->t5sReasonLayers,
"FF"->t5sReasonFF,
"Dropout"->t5sReasonDropout,
"LearningRate"->t5sReasonLR,
"BatchSize"->t5sReasonBatch,
"Rounds"->t5sReasonRounds,
"ValidationBalancedGate"->t5sReasonValidationBalancedGate,
"TrainingBalancedGate"->t5sReasonTrainingBalancedGate
|>;
t5sPerceptionSpec=<|
"DModel"->t5sPerceptionDModel,
"Heads"->t5sPerceptionHeads,
"Layers"->t5sPerceptionLayers,
"FF"->t5sPerceptionFF,
"Dropout"->t5sPerceptionDropout,
"LearningRate"->t5sPerceptionLR,
"BatchSize"->t5sPerceptionBatch,
"Rounds"->t5sPerceptionRounds
|>;
t5sPreWorldProtocolHash=Hash[
{discoveryPolicySpecS119B,t5sReasonSpec,t5sPerceptionSpec,t5sWinnerMargin},
"SHA256",
"HexString"
];
frozenDiscoveryPolicyHashS119B=Hash[discoveryPolicySpecS119B,"SHA256","HexString"];
Print["PublicActions=",publicActionsS119B];
Print["PublicProbes=",publicProbesS119B];
Print["MaximumTrainingInteractionOrder=2"];
Print["FreshHiddenWorldSeed=",hiddenBenchmarkSeedS119B];
Print["PreWorldProtocolHash=",t5sPreWorldProtocolHash];
Print["HIDDEN WORLD DOES NOT EXIST YET."];
BlockRandom[
SeedRandom[t5sPerceptionPrototypeSeed];

```

```

t5sPerceptionBases=Normalize/@RandomVariate[
NormalDistribution[0.,1.],
{2,t5sPerceptionSensorDim}
]
];
T5SPerceptionPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t5sPerceptionPosDim))],
Cos[pos/(10000.^(2.0*k/t5sPerceptionPosDim))]}
],
{k,0,t5sPerceptionPosDim/2-1}
]
];
T5SBinarySensory[y_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
Module[{drift,gain},
drift=RandomReal[
{-t5sPerceptionDrift,t5sPerceptionDrift},
t5sPerceptionSensorDim
];
N[
Table[
gain=1.+RandomReal[
{-t5sPerceptionGainJitter,t5sPerceptionGainJitter}
];
Join[
t5sPerceptionScale*gain*t5sPerceptionBases[[y+1]]+
drift+
RandomVariate[
NormalDistribution[0.,t5sPerceptionNoise],
t5sPerceptionSensorDim
],
T5SPerceptionPos[m]
],
{m,1,t5sPerceptionMicroLen}
]
];
t5sPerceptionInputDim=t5sPerceptionSensorDim+t5sPerceptionPosDim;
T5SMakePerceptionData[n_Integer,seed_Integer]:=
BlockRandom[
SeedRandom[seed];
RandomSample[
Flatten[
Table[
Table[
<|
"Input"->T5SBinarySensory[y,seed+100000*y+i],
"Label"->y
|>,
{i,1,n}
],
]
];

```

```

{y,0,1}
],
1
]
]
];
T5SPerceptionBlock[modelDim_Integer,heads_Integer,ffDim_Integer,drop_?NumericQ]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer[
"Dot",
"MultiHead"->True,
"Mask"->"Causal",
"ScoreRescaling"->"DimensionSqrt",
"Dropout"->drop
],
"Merge"->NetMapOperator[
NetChain[{FlattenLayer[],LinearLayer[modelDim]}]
],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2,"Same"],
"FF"->NetMapOperator[
NetChain[
{
LinearLayer[ffDim],
ElementwiseLayer[Ramp],
DropoutLayer[drop],
LinearLayer[modelDim]
}
]
],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention","Query"],
"K"->NetPort["Attention","Key"],
"V"->NetPort["Attention","Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{NetPort["Input"],"ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1","FDrop"}->"Res2"
}

```

```

},
"Input"->{"Varying",modelDim}
]
];
T5SPerceptionNet[]:=
Module[{blocks},
blocks=Table[
T5SPerceptionBlock[
t5sPerceptionDModel,
t5sPerceptionHeads,
t5sPerceptionFF,
t5sPerceptionDropout
],
{t5sPerceptionLayers}
];
NetChain[
Join[
{NetMapOperator[LinearLayer[t5sPerceptionDModel]]},
blocks,
{
NormalizationLayer[2,"Same"],
SequenceLastLayer[],
LinearLayer[2],
SoftmaxLayer[]
}
],
"Input"->{"Varying",t5sPerceptionInputDim},
"Output"->NetDecoder[{"Class",{0,1}}]
]
];
Print["====="];
Print["PHASE 1: TRAIN AND FREEZE SHARED PERCEPTION"];
Print["HIDDEN WORLD STILL DOES NOT EXIST"];
Print["====="];
t5sPerceptionTrain=T5SMakePerceptionData[
t5sPerceptionTrainPerClass,
t5sPerceptionTrainSeed
];
t5sPerceptionVal=T5SMakePerceptionData[
t5sPerceptionValPerClass,
t5sPerceptionValSeed
];
t5sPerceptionTrainRules=(#[ "Input" ]->#[ "Label" ]&)/@t5sPerceptionTrain;
t5sPerceptionValRules=(#[ "Input" ]->#[ "Label" ]&)/@t5sPerceptionVal;
SeedRandom[t5sPerceptionNetSeed];
t5sPerceptionNet0=NetInitialize[T5SPerceptionNet[]];
t5sPerceptionTraining=
NetTrain[
t5sPerceptionNet0,
t5sPerceptionTrainRules,
All,
ValidationSet->t5sPerceptionValRules,
MaxTrainingRounds->t5sPerceptionRounds,
BatchSize->t5sPerceptionBatch,
LearningRate->t5sPerceptionLR,
Method->"ADAM",

```

```

TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t5sPerceptionPatience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t5sTargetDevice,
RandomSeeding->t5sPerceptionNetSeed
];
t5sFrozenPerception=t5sPerceptionTraining["TrainedNet"];
t5sPerceptionValidationAccuracy=
NetMeasurements[
t5sFrozenPerception,
t5sPerceptionValRules,
"Accuracy",
BatchSize->t5sPerceptionBatch
];
Print["PerceptionValidationAccuracy=", t5sPerceptionValidationAccuracy];
If[
!NumericQ[t5sPerceptionValidationAccuracy]||
t5sPerceptionValidationAccuracy<t5sPerceptionGate,
T5SStop["shared perception failed validation gate"]
];
Export[
FileNameJoin[
{t5sOutputDirectory, "S124_T5R1_PERCEPTION_FROZEN_BEFORE_WORLD.wlnet"}
],
t5sFrozenPerception
];
Clear[
t5sPerceptionTrain,
t5sPerceptionVal,
t5sPerceptionTrainRules,
t5sPerceptionValRules,
t5sPerceptionNet0,
t5sPerceptionTraining
];
Print["SHARED PERCEPTION FROZEN"];
Print["====="];
Print["PHASE 2: GENERATE FRESH S119B HIDDEN WORLD"];
Print["====="];
hiddenFactorSizesS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B];
RandomSample[{2,3,4,5}]
];
hiddenTrueFactorCountS119B=Length[hiddenFactorSizesS119B];
hiddenSourceFactorsS119B=Flatten[
Position[hiddenFactorSizesS119B,2|3]
];
hiddenTargetFactorsS119B=Flatten[
Position[hiddenFactorSizesS119B,4|5]
];
hiddenSourceFactorsS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+1];

```

```

RandomSample[hiddenSourceFactorsS119B]
];
hiddenTargetFactorsS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+2];
RandomSample[hiddenTargetFactorsS119B]
];
hiddenCouplingPairsS119B=
Thread[hiddenSourceFactorsS119B->hiddenTargetFactorsS119B];
hiddenActionRolePoolsS119B=
Flatten[
Table[
Module[{parentForPlus},
parentForPlus=
Lookup[
Association[Reverse/@hiddenCouplingPairsS119B],
factorIndex,
0
];
{
{factorIndex,1,parentForPlus},
{factorIndex,-1,0}
}
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
hiddenActionRolePermutationS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+3];
RandomSample[hiddenActionRolePoolsS119B]
];
hiddenActionRolesS119B=
AssociationThread[
publicActionsS119B->
hiddenActionRolePermutationS119B
];
hiddenProbeRolePoolsS119B=
Flatten[
Table[
Table[
{factorIndex,value},
{value,0,hiddenFactorSizesS119B[[factorIndex]]-1}
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
hiddenProbeRolePermutationS119B=
BlockRandom[
SeedRandom[hiddenBenchmarkSeedS119B+4];
RandomSample[hiddenProbeRolePoolsS119B]
];
hiddenProbeRolesS119B=
AssociationThread[

```

```

publicProbesS119B->
hiddenProbeRolePermutationS119B
];
hiddenStartStateS119B=
ConstantArray[0,hiddenTrueFactorCountS119B];
S119BEnvStep[state_List,action_Integer]:=
Module[
{role,targetFactor,direction,parentFactor,targetSize,delta,nextState},
role=Lookup[
hiddenActionRolesS119B,
action,
Missing["UnknownAction"]
];
If[
!ListQ[role],
Missing["UnknownAction"],
targetFactor=role[[1]];
direction=role[[2]];
parentFactor=role[[3]];
targetSize=hiddenFactorSizesS119B[[targetFactor]];
delta=Which[
parentFactor==0,direction,
direction==1&&state[[parentFactor]]==0,1,
direction==1&&state[[parentFactor]]!=0,2,
True,direction
];
nextState=state;
nextState[[targetFactor]]=
Mod[
nextState[[targetFactor]]+delta,
targetSize
];
nextState
]
];
S119BEnvStateAfter[seq_List]:=
Fold[
S119BEnvStep,
hiddenStartStateS119B,
seq
];
S119BEnvRawOutput[seq_List,probe_Integer]:=
Module[{state,role,factorIndex,value},
state=S119BEnvStateAfter[seq];
role=Lookup[
hiddenProbeRolesS119B,
probe,
Missing["UnknownProbe"]
];
If[
!ListQ[state]||!ListQ[role],
Missing["EnvironmentFailure"],
factorIndex=role[[1]];
value=role[[2]];
Boole[state[[factorIndex]]==value]
]
]

```

```

];
hiddenIndependentMinusActionByFactorS119B=
Association[
Table[
factorIndex->
SelectFirst[
publicActionsS119B,
Function[
action,
Lookup[hiddenActionRolesS119B,action]=={factorIndex,-1,0}
]
],
{factorIndex,1,hiddenTrueFactorCountS119B}
];
S119BEvaluatorCanonicalSequence[tuple_List]:=
Flatten[
Table[
ConstantArray[
Lookup[
hiddenIndependentMinusActionByFactorS119B,
factorIndex
],
Mod[
-tuple[[factorIndex]],
hiddenFactorSizesS119B[[factorIndex]]
]
],
{factorIndex,1,hiddenTrueFactorCountS119B}
],
1
];
allTrueJointTuplesS119B=
Tuples[
Map[
Range[0,#-1]&,
hiddenFactorSizesS119B
]
];
trueJointStateCountS119B=
Length[allTrueJointTuplesS119B];
prospectiveHighOrderTuplesS119B=
Select[
allTrueJointTuplesS119B,
Count[#,Except[0]]>=3&
];
prospectiveHighOrderHoldoutS119B=
Table[
<|
"TupleEvaluatorOnly"->tuple,
"Sequence"->S119BEvaluatorCanonicalSequence[tuple]
|>,
{tuple,prospectiveHighOrderTuplesS119B}
];
Print["FreshFactorSizesEvaluatorOnly=",hiddenFactorSizesS119B];
Print["TrueJointStates=",trueJointStateCountS119B];

```

```

Print["HighOrderHoldoutStates=",Length[prospectiveHighOrderHoldoutS119B]];
Print["HIGH-ORDER OUTPUTS REMAIN SEALED"];
S119BSeqKey[seq_List]:=
ToString[InputForm[seq]];
S119BQueryKey[seq_List,probe_Integer]:=
ToString[InputForm[{seq,probe}]];
T5SQuerySensorSeed[seq_List,probe_Integer]:=
1+Mod[
Hash[
{seq,probe,t5sPerceptionQuerySeed},
"CRC32"
],
2000000000
];
membershipCacheS119B=<||>;
queriedSequenceRegistryS119B=<||>;
membershipUniqueCountS119B=0;
t5sMembershipRows={};
S119BMembershipY[
seq_List,
probe_Integer,
source_String:"SupportScan"
]:=
Module[{key,seqKey,rawY,perceivedY},
key=S119BQueryKey[seq,probe];
seqKey=S119BSeqKey[seq];
If[
KeyExistsQ[membershipCacheS119B,key],
Lookup[membershipCacheS119B,key],
rawY=S119BEnvRawOutput[seq,probe];
If[
!MemberQ[{0,1},rawY],
T5SStop["environment returned non-binary output"]
];
perceivedY=
t5sFrozenPerception[
T5SBinarySensory[
rawY,
T5SQuerySensorSeed[seq,probe]
]
];
If[
!MemberQ[{0,1},perceivedY],
T5SStop["perception returned invalid class"]
];
AssociateTo[membershipCacheS119B,key->perceivedY];
AssociateTo[queriedSequenceRegistryS119B,seqKey->seq];
membershipUniqueCountS119B++;
AppendTo[
t5sMembershipRows,
<|
"Seq"->seq,
"Probe"->probe,
"Y"->perceivedY,
"RawYEvaluatorOnly"->rawY,
"Source"->source

```

```

|>
];
perceivedY
]
];
S119BObserveSignature[
seq_List,
probes_List,
source_String
]:=
Table[
S119BMembershipY[
seq,
probe,
source
],
{probe,probes}
];
S119BSignatureKey[sig_List]:=
ToString[InputForm[sig]];
Print["====="];
Print["PHASE 3: FACTOR SUPPORT DISCOVERY"];
Print["HIGH-ORDER OUTPUTS SEALED"];
Print["====="];
baselineSignatureS119B=
S119BObserveSignature[
{},
publicProbesS119B,
"SupportScan"
];
actionSupportRowsS119B=
Table[
Module[{scanSignatures,changedPositions,support},
scanSignatures=
Table[
S119BObserveSignature[
ConstantArray[action,repetition],
publicProbesS119B,
"SupportScan"
],
{repetition,1,supportScanDepthS119B}
];
changedPositions=
Select[
Range[Length[publicProbesS119B]],
Function[
position,
AnyTrue[
scanSignatures,
Function[
signature,
signature[[position]]!=baselineSignatureS119B[[position]]
]
]
]
];
];

```

```

support=publicProbesS119B[[changedPositions]];
<|
"Action"->action,
"ObservedProbeSupport"->Sort[support],
"SupportSize"->Length[support]
|>
],
{action,publicActionsS119B}
];
If[
AnyTrue[
actionSupportRowsS119B,
Length[#["ObservedProbeSupport"]]==0&
],
T5SStop["empty factor support"]
];
supportGroupsS119B=
Values[
GroupBy[
actionSupportRowsS119B,
ToString[
InputForm[
#["ObservedProbeSupport"]
]
]&
]
];
factorDescriptorsS119B=
Table[
<|
"Probes"->Sort[
supportGroupsS119B[[g,1]]["ObservedProbeSupport"]
],
"Actions"->Sort[
(#["Action"]&)/@supportGroupsS119B[[g]]
]
|>,
{g,1,Length[supportGroupsS119B]}
];
factorDescriptorsS119B=
SortBy[
factorDescriptorsS119B,
First[#["Probes"]]&
];
factorDescriptorsS119B=
Table[
Join[
factorDescriptorsS119B[[i]],
<| "LearnedFactor" -> i |>
],
{i,1,Length[factorDescriptorsS119B]}
];
inferredFactorCountS119B=
Length[factorDescriptorsS119B];
learnedProbeGroupsS119B=
(#["Probes"]&)/@factorDescriptorsS119B;

```

```

learnedActionGroupsS119B=
(#["Actions"]&)/@factorDescriptorsS119B;
Print["InferredFactors=",inferredFactorCountS119B];
Print["ProbeGroupSizes=",Length/@learnedProbeGroupsS119B];
Print["ActionGroupSizes=",Length/@learnedActionGroupsS119B];
If[
inferredFactorCountS119B!=4,
T5SStop["failed to recover four factors"]
];
S119BLocalTransitionKey[state_Integer,action_Integer]:=
ToString[InputForm[{state,action}]];
S119BLearnLocalFactor[descriptor_Association]:=
Module[
{
probes,actions,startSignature,signatureToState,
stateSignatures,representatives,transitions,queue,
currentState,currentRep,nextSeq,nextSignature,nextKey,
nextState,newState,guard
},
probes=descriptor["Probes"];
actions=descriptor["Actions"];
startSignature=
S119BObserveSignature[
{},
probes,
"LocalBFS"
];
signatureToState=
<|S119BSignatureKey[startSignature]->1|>;
stateSignatures=<|1->startSignature|>;
representatives=<|1->{}|>;
transitions=<||>;
queue={1};
guard=0;
While[
Length[queue]>0,
guard++;
If[
guard>localBFSSafetyS119B,
T5SStop["local BFS safety exceeded"]
];
currentState=First[queue];
queue=Rest[queue];
currentRep=
Lookup[
representatives,
currentState
];
Do[
nextSeq=Append[currentRep,action];
nextSignature=
S119BObserveSignature[
nextSeq,
probes,
"LocalBFS"
];
];

```

```

nextKey=
S119BSignatureKey[nextSignature];
If[
KeyExistsQ[signatureToState,nextKey],
nextState=
Lookup[
signatureToState,
nextKey
],
newState=Length[stateSignatures]+1;
nextState=newState;
AssociateTo[
signatureToState,
nextKey->newState
];
AssociateTo[
stateSignatures,
newState->nextSignature
];
AssociateTo[
representatives,
newState->nextSeq
];
AppendTo[
queue,
newState
]
];
AssociateTo[
transitions,
S119BLocalTransitionKey[
currentState,
action
]->nextState
],
{action,actions}
]
];
<|
"LearnedFactor"->descriptor["LearnedFactor"],
"Probes"->probes,
"Actions"->actions,
"StateCount"->Length[stateSignatures],
"StateSignatures"->stateSignatures,
"RepresentativeByState"->representatives,
"TransitionsAtBaseline"->transitions,
"StartState"->1,
"ObservedOneHot"->AllTrue[
Values[stateSignatures],
Total[#]==1&
]
|>
];
learnedFactorsS119B=
S119BLearnLocalFactor/@factorDescriptorsS119B;
learnedLocalStateCountsS119B=

```

```

(#["StateCount"]&)/@learnedFactorsS119B;
Print["LocalStateCounts=",learnedLocalStateCountsS119B];
Print["StoredLocalStates=",Total[learnedLocalStateCountsS119B]];
Print["ProductCapacity=",Times@@learnedLocalStateCountsS119B];
If[
Sort[learnedLocalStateCountsS119B]!=Sort[{2,3,4,5}],
TSSStop["unexpected learned local state cardinalities"]
];
actionToLearnedFactorS119B=
Association[
Flatten[
Table[
Table[
action->factorIndex,
{
action,
learnedFactorsS119B[[factorIndex]]["Actions"]
}
],
{factorIndex,1,Length[learnedFactorsS119B]}
]
];
probeToLearnedFactorS119B=
Association[
Flatten[
Table[
Table[
probe->factorIndex,
{
probe,
learnedFactorsS119B[[factorIndex]]["Probes"]
}
],
{factorIndex,1,Length[learnedFactorsS119B]}
]
];
S119BLocalStateFromSignature[
factorIndex_Integer,
signature_List
]:=
SelectFirst[
Range[
learnedFactorsS119B[[factorIndex]]["StateCount"]
],
Function[
localState,
Lookup[
learnedFactorsS119B[[factorIndex]]["StateSignatures"],
localState
]==signature
],
Missing["UnknownLocalSignature"]
];
S119BPrepareFactorState[

```

```

baseSequence_List,
factorIndex_Integer,
desiredLocalState_Integer,
source_String
]:=
Module[{result, probes, actions, desiredSignature},
probes=
learnedFactorsS119B[[factorIndex]]["Probes"];
actions=
learnedFactorsS119B[[factorIndex]]["Actions"];
desiredSignature=
Lookup[
learnedFactorsS119B[[factorIndex]]["StateSignatures"],
desiredLocalState
];
result=
Catch[
Module[
{
startSignature, queue, visited, current,
currentSequence, currentSignature, nextSequence,
nextSignature, nextKey, guard
},
startSignature=
S119BObserveSignature[
baseSequence,
probes,
source
];
If[
startSignature===desiredSignature,
Throw[
baseSequence,
"S119B_PREPARED"
]
];
queue={{baseSequence, startSignature}};
visited=
<|S119BSignatureKey[startSignature]->True|>;
guard=0;
While[
Length[queue]>0,
guard++;
If[
guard>preparationBFSSafetyS119B,
Throw[
$Failed,
"S119B_PREPARED"
]
];
current=First[queue];
queue=Rest[queue];
currentSequence=current[[1]];
currentSignature=current[[2]];
Do[
nextSequence=

```

```

Append[
currentSequence,
action
];
nextSignature=
S119BObserveSignature[
nextSequence,
probes,
source
];
If[
nextSignature===desiredSignature,
Throw[
nextSequence,
"S119B_PREPARED"
]
];
nextKey=
S119BSignatureKey[nextSignature];
If[
!KeyExistsQ[visited,nextKey],
AssociateTo[
visited,
nextKey->True
];
AppendTo[
queue,
{nextSequence,nextSignature}
],
{action,actions}
];
$Failed
],
"S119B_PREPARED"
];
If[
result=== $Failed,
T5SStop["could not prepare requested factor state"],
result
];
];
S119BPrepareContext[
assignments_List,
source_String
]:=
Module[{seq={}},
Do[
seq=
S119BPrepareFactorState[
seq,
assignment[[1]],
assignment[[2]],
source
],

```

```

{assignment,assignments}
];
seq
];
Print["====="];
Print["PHASE 4: ACTIVE PAIRWISE INTERACTION DISCOVERY"];
Print["HIGH-ORDER OUTPUTS SEALED"];
Print["====="];
interactionScanRowsS119B=
Flatten[
Table[
Module[
{targetFactor,seq,nextSeq,targetSignature,destinationState},
targetFactor=
Lookup[
actionToLearnedFactorS119B,
action
];
seq=
S119BPrepareContext[
{
{targetFactor,targetState},
{candidateParent,parentState}
},
"InteractionScan"
];
nextSeq=Append[seq,action];
targetSignature=
S119BObserveSignature[
nextSeq,
learnedFactorsS119B[[targetFactor]][ "Probes" ],
"InteractionScan"
];
destinationState=
S119BLocalStateFromSignature[
targetFactor,
targetSignature
];
If[
!IntegerQ[destinationState],
T5SSStop["interaction scan reached unknown state"]
];
<|
"Action"->action,
"TargetFactor"->targetFactor,
"CandidateParent"->candidateParent,
"TargetState"->targetState,
"ParentState"->parentState,
"DestinationState"->destinationState
|>
],
{action,publicActionsS119B},
{
candidateParent,
DeleteCases[
Range[inferredFactorCountS119B],

```

```

Lookup[actionToLearnedFactorS119B,action]
]
},
{
targetState,
Range[
learnedFactorsS119B[[
Lookup[actionToLearnedFactorS119B,action]
]]["StateCount"]
]
},
{
parentState,
Range[
learnedFactorsS119B[[candidateParent]]["StateCount"]
]
}
],
3
];
S119BDependencyDetectedQ[
action_Integer,
candidateParent_Integer
]:=
Module[{rows,targetFactor,targetStateCount},
rows=
Select[
interactionScanRowsS119B,
#[["Action"]==action&&
#[["CandidateParent"]==candidateParent&
];
targetFactor=
Lookup[
actionToLearnedFactorS119B,
action
];
targetStateCount=
learnedFactorsS119B[[targetFactor]]["StateCount"];
AnyTrue[
Range[targetStateCount],
Function[
targetState,
Length[
DeleteDuplicates[
(#[["DestinationState"]&)/@
Select[
rows,
#[["TargetState"]==targetState&
]
]
]>1
]
]
];
learnedParentsByActionS119B=
Association[

```

```

Table[
action->
Sort[
Select[
DeleteCases[
Range[inferredFactorCountS119B],
Lookup[actionToLearnedFactorS119B,action]
],
S119BDependencyDetectedQ[
action,
#
]&
]
],
{action,publicActionsS119B}
]
];
learnedInteractionEdgesS119B=
DeleteDuplicates[
Flatten[
Table[
Module[{targetFactor},
targetFactor=
Lookup[
actionToLearnedFactorS119B,
action
];
Table[
parent->targetFactor,
{
parent,
Lookup[
learnedParentsByActionS119B,
action
]
}
],
{action,publicActionsS119B}
],
1
]
];
Print["LearnedParentsByAction=",learnedParentsByActionS119B];
Print["LearnedInteractionEdges=",learnedInteractionEdgesS119B];
If[
Length[learnedInteractionEdgesS119B]!=2,
T5SStop["expected exactly two sparse interaction edges"]
];
S119BConditionalTransitionKey[
action_Integer,
targetState_Integer,
parentStates_List
]:=
ToString[
InputForm[

```

```

{action,targetState,parentStates}
]
];
conditionalTransitionsS119B=<|>;
conditionalTransitionRowsS119B={};
Do[
Module[
{
targetFactor,parentFactors,parentStateTuples,
destination,seq,nextSeq,targetSignature,assignments
},
targetFactor=
Lookup[
actionToLearnedFactorS119B,
action
];
parentFactors=
Lookup[
learnedParentsByActionS119B,
action
];
If[
Length[parentFactors]==0,
Do[
destination=
Lookup[
learnedFactorsS119B[[targetFactor]]["TransitionsAtBaseline"],
S119BLocalTransitionKey[
targetState,
action
]
];
AssociateTo[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[
action,
targetState,
{}
]->destination
];
AppendTo[
conditionalTransitionRowsS119B,
<|
"Action"->action,
"TargetFactor"->targetFactor,
"ParentFactors"->{},
"TargetState"->targetState,
"ParentStates"->{},
"DestinationState"->destination
|>
],
{
targetState,
1,
learnedFactorsS119B[[targetFactor]]["StateCount"]
}
}

```

```

],
parentStateTuples=
Tuples[
Table[
Range[
learnedFactorsS119B[[parentFactor]][["StateCount"]
],
{parentFactor,parentFactors}
]
];
Do[
Do[
assignments=
Join[
{{targetFactor,targetState}},
MapThread[
List,
{parentFactors,parentStates}
]
];
seq=
S119BPrepareContext[
assignments,
"ConditionalTable"
];
nextSeq=Append[seq,action];
targetSignature=
S119BObserveSignature[
nextSeq,
learnedFactorsS119B[[targetFactor]][["Probes"]],
"ConditionalTable"
];
destination=
S119BLocalStateFromSignature[
targetFactor,
targetSignature
];
If[
!IntegerQ[destination],
T5SStop["conditional table reached unknown state"]
];
AssociateTo[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[
action,
targetState,
parentStates
]->destination
];
AppendTo[
conditionalTransitionRowsS119B,
<|
"Action"->action,
"TargetFactor"->targetFactor,
"ParentFactors"->parentFactors,
"TargetState"->targetState,

```

```

"ParentStates"->parentStates,
"DestinationState"->destination
|>
],
{parentStates,parentStateTuples}
],
{
targetState,
1,
learnedFactorsS119B[[targetFactor]][ "StateCount" ]
}
]
]
],
{action,publicActionsS119B}
];
Print["ConditionalTransitionCells=",Length[conditionalTransitionsS119B]];
If[
Length[conditionalTransitionsS119B]!=42,
T5SStop["expected 42 sparse conditional transition cells"]
];
factorizedStartStateS119B=
(#["StartState"]&)/@learnedFactorsS119B;
S119BFactorizedStep[
jointState_List,
action_Integer
]:=
Module[
{
targetFactor,parentFactors,targetState,
parentStates,destination,nextState
},
targetFactor=
Lookup[
actionToLearnedFactorS119B,
action,
Missing["UnknownAction"]
];
If[
!IntegerQ[targetFactor],
Missing["UnknownAction"],
parentFactors=
Lookup[
learnedParentsByActionS119B,
action,
{}
];
targetState=
jointState[[targetFactor]];
parentStates=
If[
Length[parentFactors]==0,
{},
jointState[[parentFactors]]
];
destination=

```

```

Lookup[
conditionalTransitionsS119B,
S119BConditionalTransitionKey[
action,
targetState,
parentStates
],
Missing["UnknownConditionalTransition"]
];
If[
!IntegerQ[destination],
Missing["UnknownConditionalTransition"],
nextState=jointState;
nextState[[targetFactor]]=destination;
nextState
]
]
];
S119BFactorizedStateAfter[seq_List]:=
Fold[
Function[
{state,action},
If[
ListQ[state],
S119BFactorizedStep[state,action],
state
]
],
factorizedStartStateS119B,
seq
];
S119BFactorizedOutput[
seq_List,
probe_Integer
]:=
Module[
{
state,factorIndex,localState,
localSignature,probePosition
},
state=
S119BFactorizedStateAfter[seq];
factorIndex=
Lookup[
probeToLearnFactorS119B,
probe,
Missing["UnknownProbe"]
];
If[
!ListQ[state]||!IntegerQ[factorIndex],
Missing["ModelFailure"],
localState=
state[[factorIndex]];
localSignature=
Lookup[
learnedFactorsS119B[[factorIndex]]["StateSignatures"],

```

```

localState,
Missing["UnknownState"]
];
probePosition=
FirstPosition[
learnedFactorsS119B[[factorIndex]]["Probes"],
probe,
Missing["UnknownProbe"]
];
If[
!ListQ[localSignature]||
MissingQ[probePosition],
Missing["ModelFailure"],
localSignature[[probePosition[[1]]]]
]
];
highOrderTouchedBeforeFreezeS119B=
Count[
prospectiveHighOrderHoldoutS119B,
row_Association/;
KeyExistsQ[
queriedSequenceRegistryS119B,
S119BSeqKey[row["Sequence"]]
]
];
membershipQueriesBeforeFreezeS119B=
membershipUniqueCountS119B;
policyHashUnchangedS119B=
Hash[
discoveryPolicySpecS119B,
"SHA256",
"HexString"
]==frozenDiscoveryPolicyHashS119B;
If[
!TrueQ[policyHashUnchangedS119B],
T5SStop["frozen discovery policy changed"]
];
t5sTrainingPerceptionAccuracy=
N[
Mean[
Boole[
#[ "Y" ]==#[ "RawYEvaluatorOnly" ]
]&/@t5sMembershipRows
]
];
t5sTrainingUniqueSequences=
DeleteDuplicatesBy[
t5sMembershipRows,
S119BSeqKey[#[ "Seq" ]]&
];
t5sMaximumTrainingInteractionOrder=
Max[
Count[
S119BEnvStateAfter[#[ "Seq" ]],
Except[0]
]
]

```

```

]&/@t5sTrainingUniqueSequences
];
Print["TrainingMembershipPerceptionAccuracy=",t5sTrainingPerceptionAccuracy];
Print["MaximumTrainingInteractionOrder=",t5sMaximumTrainingInteractionOrder];
Print["MembershipQueriesBeforeFreeze=",membershipQueriesBeforeFreezeS119B];
Print["HighOrderTouchedBeforeFreeze=",highOrderTouchedBeforeFreezeS119B];
If[
t5sTrainingPerceptionAccuracy<t5sTrainingPerceptionGate,
T5SStop["membership perception competence gate failed"]
];
If[
t5sMaximumTrainingInteractionOrder>2,
T5SStop["training exceeded pairwise interaction order"]
];
If[
highOrderTouchedBeforeFreezeS119B!=0,
T5SStop["high-order holdout leakage detected"]
];
frozenModelHashS119B=
Hash[
{
factorDescriptorsS119B,
learnedFactorsS119B,
actionToLearnedFactorS119B,
learnedParentsByActionS119B,
conditionalTransitionsS119B,
factorizedStartStateS119B
},
"SHA256",
"HexString"
];
Put[
<|
"Factors"->factorDescriptorsS119B,
"LearnedFactors"->learnedFactorsS119B,
"Parents"->learnedParentsByActionS119B,
"ConditionalTransitions"->conditionalTransitionsS119B,
"StartState"->factorizedStartStateS119B,
"Hash"->frozenModelHashS119B
|>,
FileNameJoin[
{t5sOutputDirectory,"S124_T5R1_TCCT_FROZEN_BEFORE_HIGHORDER.wl"}
]
];
Print["TCCT MODEL FROZEN"];
Print["TCCTFreezeHash=",frozenModelHashS119B];
Print["====="];
Print["PHASE 5: MATCHED NEURAL REASONER"];
Print["USES SAME S119B MEMBERSHIP DATA"];
Print["HIGH-ORDER OUTPUTS SEALED"];
Print["====="];
T5SReasonPos[pos_Integer]:=
Flatten[
Table[
{
Sin[pos/(10000.^(2.0*k/t5sReasonPosDim))],

```

```

Cos[pos/(10000.^(2.0*k/t5sReasonPosDim))]
},
{k,0,t5sReasonPosDim/2-1}
]
];
t5sReasonTrainingMaxLength=
Max[
Length/@Lookup[
t5sMembershipRows,
"Seq"
]
];
t5sReasonMaxSeqLen=
Max[
16,
t5sReasonTrainingMaxLength+4
];
t5sReasonInputDim=
8+14+3+t5sReasonPosDim;
T5SReasonActionToken[
action_Integer,
pos_Integer
]:=
Join[
UnitVector[8,action+1],
ConstantArray[0.,14],
{1.,0.,0.},
T5SReasonPos[pos]
];
T5SReasonPadToken[pos_Integer]:=
Join[
ConstantArray[0.,8],
ConstantArray[0.,14],
{0.,1.,0.},
T5SReasonPos[pos]
];
T5SReasonProbeToken[
probe_Integer,
pos_Integer
]:=
Join[
ConstantArray[0.,8],
UnitVector[14,probe+1],
{0.,0.,1.},
T5SReasonPos[pos]
];
T5SReasonInput[
seq_List,
probe_Integer
]:=
Module[{padded,tokens},
If[
Length[seq]>t5sReasonMaxSeqLen,
T5SStop["reason sequence exceeds frozen capacity"]
];
padded=

```

```

PadRight[
seq,
t5sReasonMaxSeqLen,
-1
];
tokens=
MapIndexed[
If[
#1===-1,
T5SReasonPadToken[First[#2]],
T5SReasonActionToken[#1,First[#2]]
]&,
padded
];
N[
Append[
tokens,
T5SReasonProbeToken[
probe,
t5sReasonMaxSeqLen+1
]
]
];
t5sReasonSample=
T5SReasonInput[
First[t5sMembershipRows]["Seq"],
First[t5sMembershipRows]["Probe"]
];
Print["ReasonInputDimensions=",Dimensions[t5sReasonSample]];
Print["ReasonMaxSequenceLength=",t5sReasonMaxSeqLen];
If[
Dimensions[t5sReasonSample]!={t5sReasonMaxSeqLen+1,t5sReasonInputDim},
T5SStop["reason input shape audit failed"]
];
If[
!MatrixQ[t5sReasonSample,NumericQ],
T5SStop["reason input is not numeric matrix"]
];
T5SReasonBlock[
modelDim_Integer,
heads_Integer,
ffDim_Integer,
drop_?NumericQ
]:=
Module[{hd},
hd=Quotient[modelDim,heads];
NetGraph[
<|
"LN1"->NormalizationLayer[2,"Same"],
"Q"->NetMapOperator[LinearLayer[{heads,hd}]],
"K"->NetMapOperator[LinearLayer[{heads,hd}]],
"V"->NetMapOperator[LinearLayer[{heads,hd}]],
"Attention"->AttentionLayer[
"Dot",
"MultiHead"->True,

```

```

"Mask"->"Causal",
"ScoreRescaling"->"DimensionSqrt",
"Dropout"->drop
],
"Merge"->NetMapOperator[
NetChain[{FlattenLayer[], LinearLayer[modelDim]}]
],
"ADrop"->DropoutLayer[drop],
"Res1"->ThreadingLayer[Plus],
"LN2"->NormalizationLayer[2, "Same"],
"FF"->NetMapOperator[
NetChain[
{
LinearLayer[ffDim],
ElementwiseLayer[Ramp],
DropoutLayer[drop],
LinearLayer[modelDim]
}
]
],
"FDrop"->DropoutLayer[drop],
"Res2"->ThreadingLayer[Plus]
|>,
{
NetPort["Input"]->"LN1",
"LN1"->"Q",
"LN1"->"K",
"LN1"->"V",
"Q"->NetPort["Attention", "Query"],
"K"->NetPort["Attention", "Key"],
"V"->NetPort["Attention", "Value"],
"Attention"->"Merge",
"Merge"->"ADrop",
{NetPort["Input"], "ADrop"}->"Res1",
"Res1"->"LN2",
"LN2"->"FF",
"FF"->"FDrop",
{"Res1", "FDrop"}->"Res2"
},
"Input"->{"Varying", modelDim}
]
];
T5SReasonNet[]:=
Module[{blocks},
blocks=
Table[
T5SReasonBlock[
t5sReasonDModel,
t5sReasonHeads,
t5sReasonFF,
t5sReasonDropout
],
{t5sReasonLayers}
];
NetChain[
Join[

```

```

{NetMapOperator[LinearLayer[t5sReasonDModel]]},
blocks,
{
NormalizationLayer[2,"Same"],
SequenceLastLayer[],
LinearLayer[2],
SoftmaxLayer[]
}
],
"Input"->{"Varying",t5sReasonInputDim},
"Output"->NetDecoder[{"Class",{0,1}}]
]
];
T5SReasonInputs[rows_List]:=
(T5SReasonInput[
#["Seq"],
#["Probe"]
]&)/@rows;
T5SReasonLabels[rows_List]:=
Lookup[rows,"Y"];
T5SClassAccuracy[
truth_List,
pred_List,
c_Integer
]:=
Module[{idx},
idx=Flatten[Position[truth,c]];
If[
idx==={},
Indeterminate,
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{pred[[idx]],truth[[idx]]}
]
]
]
];
T5SBalancedAccuracy[
truth_List,
pred_List
]:=
Module[{a0,a1},
a0=T5SClassAccuracy[truth,pred,0];
a1=T5SClassAccuracy[truth,pred,1];
If[
NumericQ[a0]&&NumericQ[a1],
N[(a0+a1)/2.],
Indeterminate
]
];
T5SPredictBatched[
net_,
inputs_List,

```

```

batch_Integer
]:=
Module[{out={},i,last,z},
Do[
last=Min[
i+batch-1,
Length[inputs]
];
z=
net[
Take[
inputs,
{i,last}
]
];
If[!ListQ[z],z={z}];
out=Join[out,z],
{i,1,Length[inputs],batch}
];
out
];
t5sRowsBySequence=
GroupBy[
t5sMembershipRows,
S119BSeqKey[#["Seq"]]&
];
t5sSequenceKeys=
Keys[t5sRowsBySequence];
t5sShuffledKeys=
BlockRandom[
SeedRandom[t5sReasonSplitSeed];
RandomSample[t5sSequenceKeys]
];
t5sValSequenceCount=
Max[
1,
Round[
0.20*Length[t5sShuffledKeys]
]
];
t5sValKeys=
Take[
t5sShuffledKeys,
t5sValSequenceCount
];
t5sTrainKeys=
Drop[
t5sShuffledKeys,
t5sValSequenceCount
];
t5sReasonTrainRows=
Flatten[
Lookup[
t5sRowsBySequence,
t5sTrainKeys
],

```

```

1
];
t5sReasonValRows=
Flatten[
Lookup[
t5sRowsBySequence,
t5sValKeys
],
1
];
t5sReasonTrainInputs=
T5SReasonInputs[
t5sReasonTrainRows
];
t5sReasonTrainLabels=
T5SReasonLabels[
t5sReasonTrainRows
];
t5sReasonValInputs=
T5SReasonInputs[
t5sReasonValRows
];
t5sReasonValLabels=
T5SReasonLabels[
t5sReasonValRows
];
t5sReasonTrainRules=
MapThread[
Rule,
{
t5sReasonTrainInputs,
t5sReasonTrainLabels
}
];
t5sReasonValRules=
MapThread[
Rule,
{
t5sReasonValInputs,
t5sReasonValLabels
}
];
Print["ReasonTrainingRows=",Length[t5sReasonTrainRows]];
Print["ReasonValidationRows=",Length[t5sReasonValRows]];
SeedRandom[t5sReasonSelectSeed];
t5sReasonSelectNet0=
NetInitialize[
T5SReasonNet[]
];
t5sReasonSelection=
NetTrain[
t5sReasonSelectNet0,
t5sReasonTrainRules,
All,
ValidationSet->t5sReasonValRules,
MaxTrainingRounds->t5sReasonRounds,

```

```

BatchSize->t5sReasonBatch,
LearningRate->t5sReasonLR,
Method->"ADAM",
TrainingProgressMeasurements->{"Accuracy", "ErrorRate"},
TrainingStoppingCriterion-><|
"Criterion"->"Loss",
"Patience"->t5sReasonPatience
|>,
TrainingProgressReporting->"Print",
TargetDevice->t5sTargetDevice,
RandomSeeding->t5sReasonSelectSeed
];
t5sReasonSelectionNet=
t5sReasonSelection["TrainedNet"];
t5sReasonValPred=
T5SPredictBatched[
t5sReasonSelectionNet,
t5sReasonValInputs,
128
];
If[
Length[t5sReasonValPred]!=Length[t5sReasonValLabels],
T5SStop["reason validation prediction length mismatch"]
];
If[
!And@@(
MemberQ[{0,1},#]&/@t5sReasonValPred
),
T5SStop["reason validation produced non-binary prediction"]
];
t5sReasonValidationAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5sReasonValPred,
t5sReasonValLabels
}
]
]
];
t5sReasonValidationZeroAccuracy=
T5SClassAccuracy[
t5sReasonValLabels,
t5sReasonValPred,
0
];
t5sReasonValidationOneAccuracy=
T5SClassAccuracy[
t5sReasonValLabels,
t5sReasonValPred,
1
];
t5sReasonValidationBalancedAccuracy=
T5SBalancedAccuracy[

```

```

t5sReasonValLabels,
t5sReasonValPred
];
Print["ReasonValidationAccuracy=", t5sReasonValidationAccuracy];
Print["ReasonValidationZeroAccuracy=", t5sReasonValidationZeroAccuracy];
Print["ReasonValidationOneAccuracy=", t5sReasonValidationOneAccuracy];
Print["ReasonValidationBalancedAccuracy=", t5sReasonValidationBalancedAccuracy];
If[
!NumericQ[t5sReasonValidationBalancedAccuracy] ||
t5sReasonValidationBalancedAccuracy < t5sReasonValidationBalancedGate,
T5SStop["neural reasoner failed validation competence gate"]
];
t5sReasonAllInputs =
T5SReasonInputs[
t5sMembershipRows
];
t5sReasonAllLabels =
T5SReasonLabels[
t5sMembershipRows
];
t5sReasonAllRules =
MapThread[
Rule,
{
t5sReasonAllInputs,
t5sReasonAllLabels
}
];
SeedRandom[t5sReasonFinalSeed];
t5sReasonFinalNet0 =
NetInitialize[
T5SReasonNet[]
];
t5sReasonFinalTraining =
NetTrain[
t5sReasonFinalNet0,
t5sReasonAllRules,
All,
MaxTrainingRounds -> t5sReasonRounds,
BatchSize -> t5sReasonBatch,
LearningRate -> t5sReasonLR,
Method -> "ADAM",
TrainingProgressMeasurements -> {"Accuracy", "ErrorRate"},
TrainingProgressReporting -> "Print",
TargetDevice -> t5sTargetDevice,
RandomSeeding -> t5sReasonFinalSeed
];
t5sFrozenReasoner =
t5sReasonFinalTraining["TrainedNet"];
t5sReasonTrainPred =
T5SPredictBatched[
t5sFrozenReasoner,
t5sReasonAllInputs,
128
];
If[

```

```

!And@@(
MemberQ[{0,1},#]&/@t5sReasonTrainPred
),
T5SStop["final reasoner produced invalid training prediction"]
];
t5sReasonTrainingAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5sReasonTrainPred,
t5sReasonAllLabels
}
]
]
];
t5sReasonTrainingBalancedAccuracy=
T5SBalancedAccuracy[
t5sReasonAllLabels,
t5sReasonTrainPred
];
Print["FinalReasonTrainingAccuracy=",t5sReasonTrainingAccuracy];
Print["FinalReasonTrainingBalancedAccuracy=",t5sReasonTrainingBalancedAccuracy];
If[
!NumericQ[t5sReasonTrainingBalancedAccuracy]||
t5sReasonTrainingBalancedAccuracy<t5sReasonTrainingBalancedGate,
T5SStop["final reasoner failed training competence gate"]
];
t5sHighOrderMaxSequenceLength=
Max[
Length/@Lookup[
prospectiveHighOrderHoldoutS119B,
"Sequence"
]
];
If[
t5sHighOrderMaxSequenceLength+1>t5sReasonMaxSeqLen,
T5SStop["sealed holdout exceeds pre-frozen neural capacity"]
];
Export[
FileNameJoin[
{t5sOutputDirectory,"S124_T5R1_NEURAL_REASONER_FROZEN_BEFORE_HIGHORDER.wlnet"}
],
t5sFrozenReasoner
];
t5sGlobalFreezeHash=
Hash[
{
t5sPreWorldProtocolHash,
frozenModelHashS119B,
t5sPerceptionValidationAccuracy,
t5sReasonValidationBalancedAccuracy,
t5sReasonTrainingBalancedAccuracy,
t5sReasonMaxSeqLen
},
];

```

```

"SHA256",
"HexString"
];
Print["====="];
Print["GLOBAL FREEZE COMPLETE"];
Print["SharedPerceptionFrozen=True"];
Print["TCCTFrozen=True"];
Print["NeuralReasonerFrozen=True"];
Print["HighOrderOutputsOpened=False"];
Print["ReasonMaxSequenceLength=", t5sReasonMaxSeqLen];
Print["GlobalFreezeHash=", t5sGlobalFreezeHash];
Print["====="];
Clear[
t5sReasonTrainInputs,
t5sReasonTrainLabels,
t5sReasonValInputs,
t5sReasonValLabels,
t5sReasonTrainRules,
t5sReasonValRules,
t5sReasonSelection,
t5sReasonSelectNet0,
t5sReasonAllRules,
t5sReasonFinalTraining,
t5sReasonFinalNet0
];
T5SNeuralSignature[seq_List]:=
Module[{inputs,pred},
inputs=
Table[
T5SReasonInput[
seq,
probe
],
{probe,publicProbesS119B}
];
pred=
T5SPredictBatched[
t5sFrozenReasoner,
inputs,
14
];
If[
Length[pred]!=14||
!And@@(
MemberQ[{0,1},#]&/@pred
),
T5SStop["invalid neural prospective signature"]
];
pred
];
Print["====="];
Print["PHASE 6: FIRST OPENING OF FRESH HIGH-ORDER HOLDOUT"];
Print["NO MODEL CHANGES AFTER THIS LINE"];
Print["====="];
t5sHighOrderRows=
Table[

```

```

Module[
{seq,trueSignature,tcctSignature,neuralSignature},
seq=row["Sequence"];
trueSignature=
Table[
S119BEnvRawOutput[
seq,
probe
],
{probe,publicProbesS119B}
];
tcctSignature=
Table[
S119BFactorizedOutput[
seq,
probe
],
{probe,publicProbesS119B}
];
neuralSignature=
T5SNeuralSignature[
seq
];
<|
"TupleEvaluatorOnly"->row["TupleEvaluatorOnly"],
"Sequence"->seq,
"TrueSignature"->trueSignature,
"TCCTSignature"->tcctSignature,
"NeuralSignature"->neuralSignature,
"TCCTExact"->SameQ[tcctSignature,trueSignature],
"NeuralExact"->SameQ[neuralSignature,trueSignature]
|>
],
{row,prospectiveHighOrderHoldoutS119B}
];
t5sTCCTExactCount=
Count[
Lookup[
t5sHighOrderRows,
"TCCTExact"
],
True
];
t5sNeuralExactCount=
Count[
Lookup[
t5sHighOrderRows,
"NeuralExact"
],
True
];
t5sTCCTExactAccuracy=
N[
t5sTCCTExactCount/
Length[t5sHighOrderRows]
];

```

```

t5sNeuralExactAccuracy=
N[
t5sNeuralExactCount/
Length[t5sHighOrderRows]
];
t5sTrueProbeFlat=
Flatten[
Lookup[
t5sHighOrderRows,
"TrueSignature"
]
];
t5sTCCTProbeFlat=
Flatten[
Lookup[
t5sHighOrderRows,
"TCCTSignature"
]
];
t5sNeuralProbeFlat=
Flatten[
Lookup[
t5sHighOrderRows,
"NeuralSignature"
]
];
t5sTCCTProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5sTCCTProbeFlat,
t5sTrueProbeFlat
}
]
];
t5sNeuralProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5sNeuralProbeFlat,
t5sTrueProbeFlat
}
]
];
t5sTCCTProbeBalancedAccuracy=
T5SBalancedAccuracy[
t5sTrueProbeFlat,
t5sTCCTProbeFlat
];
t5sNeuralProbeZeroAccuracy=

```

```

T5SClassAccuracy[
t5sTrueProbeFlat,
t5sNeuralProbeFlat,
0
];
t5sNeuralProbeOneAccuracy=
T5SClassAccuracy[
t5sTrueProbeFlat,
t5sNeuralProbeFlat,
1
];
t5sNeuralProbeBalancedAccuracy=
T5SBalancedAccuracy[
t5sTrueProbeFlat,
t5sNeuralProbeFlat
];
Print["Opening high-order transition audit..."];
t5sHighOrderTransitionRows=
Flatten[
Table[
Module[
{seq,nextSeq,trueSignature,tcctSignature,neuralSignature},
seq=row["Sequence"];
nextSeq=Append[seq,action];
trueSignature=
Table[
S119BEnvRawOutput[
nextSeq,
probe
],
{probe,publicProbesS119B}
];
tcctSignature=
Table[
S119BFactorizedOutput[
nextSeq,
probe
],
{probe,publicProbesS119B}
];
neuralSignature=
T5SNeuralSignature[
nextSeq
];
<|
"TupleEvaluatorOnly"->row["TupleEvaluatorOnly"],
"Action"->action,
"TrueSignature"->trueSignature,
"TCCTSignature"->tcctSignature,
"NeuralSignature"->neuralSignature,
"TCCTExact"->SameQ[tcctSignature,trueSignature],
"NeuralExact"->SameQ[neuralSignature,trueSignature]
|>
],
{row,prospectiveHighOrderHoldoutS119B},
{action,publicActionsS119B}

```

```

],
1
];
t5sTCCTTransitionExactCount=
Count[
Lookup[
t5sHighOrderTransitionRows,
"TCCTExact"
],
True
];
t5sNeuralTransitionExactCount=
Count[
Lookup[
t5sHighOrderTransitionRows,
"NeuralExact"
],
True
];
t5sTCCTTransitionExactAccuracy=
N[
t5sTCCTTransitionExactCount/
Length[t5sHighOrderTransitionRows]
];
t5sNeuralTransitionExactAccuracy=
N[
t5sNeuralTransitionExactCount/
Length[t5sHighOrderTransitionRows]
];
t5sTransitionTrueFlat=
Flatten[
Lookup[
t5sHighOrderTransitionRows,
"TrueSignature"
]
];
t5sTransitionTCCTFlat=
Flatten[
Lookup[
t5sHighOrderTransitionRows,
"TCCTSignature"
]
];
t5sTransitionNeuralFlat=
Flatten[
Lookup[
t5sHighOrderTransitionRows,
"NeuralSignature"
]
];
t5sTCCTTransitionProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{

```

```

t5sTransitionTCCTFlat,
t5sTransitionTrueFlat
}
]
];
t5sNeuralTransitionProbeAccuracy=
N[
Mean[
MapThread[
Boole[SameQ[#1,#2]]&,
{
t5sTransitionNeuralFlat,
t5sTransitionTrueFlat
}
]
]
];
t5sNeuralTransitionZeroAccuracy=
T5SClassAccuracy[
t5sTransitionTrueFlat,
t5sTransitionNeuralFlat,
0
];
t5sNeuralTransitionOneAccuracy=
T5SClassAccuracy[
t5sTransitionTrueFlat,
t5sTransitionNeuralFlat,
1
];
t5sNeuralTransitionBalancedAccuracy=
T5SBalancedAccuracy[
t5sTransitionTrueFlat,
t5sTransitionNeuralFlat
];
t5sTCCTMinusNeuralExact=
N[
t5sTCCTExactAccuracy-
t5sNeuralExactAccuracy
];
t5sTCCTMinusNeuralProbe=
N[
t5sTCCTProbeAccuracy-
t5sNeuralProbeAccuracy
];
t5sTCCTMinusNeuralTransitionExact=
N[
t5sTCCTTransitionExactAccuracy-
t5sNeuralTransitionExactAccuracy
];
t5sStrictProtocolPass=
And[
t5sPerceptionValidationAccuracy>=t5sPerceptionGate,
t5sTrainingPerceptionAccuracy>=t5sTrainingPerceptionGate,
t5sMaximumTrainingInteractionOrder<=2,
highOrderTouchedBeforeFreezeS119B===0,

```

```

policyHashUnchangedS119B,
inferredFactorCountS119B==4,
Length[learnedInteractionEdgesS119B]==2,
Length[conditionalTransitionsS119B]==42,
NumericQ[t5sReasonValidationBalancedAccuracy],
t5sReasonValidationBalancedAccuracy>t5sReasonValidationBalancedGate,
NumericQ[t5sReasonTrainingBalancedAccuracy],
t5sReasonTrainingBalancedAccuracy>t5sReasonTrainingBalancedGate,
Length[t5sHighOrderRows]==74
];
t5sGeneralizationDiagnosis=
Which[
!TrueQ[t5sStrictProtocolPass],
"STRICT_PROTOCOL_GATE_FAILED",
t5sTCCTMinusNeuralExact>t5sWinnerMargin&&
t5sTCCTMinusNeuralTransitionExact>t5sWinnerMargin,
"STRICT_FRESH_WORLD_TCCT_HIGH_ORDER_GENERALIZATION_ADVANTAGE",
t5sTCCTMinusNeuralExact<=-t5sWinnerMargin&&
t5sTCCTMinusNeuralTransitionExact<=-t5sWinnerMargin,
"STRICT_FRESH_WORLD_NEURAL_HIGH_ORDER_GENERALIZATION_ADVANTAGE",
True,
"STRICT_FRESH_WORLD_GENERALIZATION_PARITY_OR_MIXED_RESULT"
];
Print[""];
Print["====="];
Print["S124-T5R1 FINAL STRICT FRESH-WORLD SUMMARY"];
Print["====="];
Print["FreshWorldSeed=",hiddenBenchmarkSeedS119B];
Print["StrictProspective=True"];
Print["SharedPerception=True"];
Print["PreWorldProtocolHash=",t5sPreWorldProtocolHash];
Print["PerceptionValidationAccuracy=",t5sPerceptionValidationAccuracy];
Print["TrainingMembershipPerceptionAccuracy=",t5sTrainingPerceptionAccuracy];
Print["MaximumTrainingInteractionOrder=",t5sMaximumTrainingInteractionOrder];
Print["TrueJointStates=",trueJointStateCountS119B];
Print["HighOrderHoldoutStates=",Length[t5sHighOrderRows]];
Print["HighOrderTouchedBeforeFreeze=",highOrderTouchedBeforeFreezeS119B];
Print["InferredFactors=",inferredFactorCountS119B];
Print["LocalStateCounts=",learnedLocalStateCountsS119B];
Print["LearnedInteractionEdges=",learnedInteractionEdgesS119B];
Print["ConditionalTransitionCells=",Length[conditionalTransitionsS119B]];
Print["MembershipQueriesBeforeFreeze=",membershipQueriesBeforeFreezeS119B];
Print["ReasonValidationAccuracy=",t5sReasonValidationAccuracy];
Print["ReasonValidationBalancedAccuracy=",t5sReasonValidationBalancedAccuracy];
Print["FinalReasonTrainingAccuracy=",t5sReasonTrainingAccuracy];
Print["FinalReasonTrainingBalancedAccuracy=",t5sReasonTrainingBalancedAccuracy];
Print["TCCTHighOrderExact=",t5sTCCTExactCount,"/",Length[t5sHighOrderRows]];
Print["TCCTHighOrderExactAccuracy=",t5sTCCTExactAccuracy];
Print["NeuralHighOrderExact=",t5sNeuralExactCount,"/",Length[t5sHighOrderRows]];
Print["NeuralHighOrderExactAccuracy=",t5sNeuralExactAccuracy];
Print["TCCTProbeAccuracy=",t5sTCCTProbeAccuracy];
Print["TCCTProbeBalancedAccuracy=",t5sTCCTProbeBalancedAccuracy];
Print["NeuralProbeAccuracy=",t5sNeuralProbeAccuracy];
Print["NeuralZeroAccuracy=",t5sNeuralProbeZeroAccuracy];
Print["NeuralOneAccuracy=",t5sNeuralProbeOneAccuracy];
Print["NeuralBalancedAccuracy=",t5sNeuralProbeBalancedAccuracy];

```

```

Print["HighOrderTransitionCases=", Length[t5sHighOrderTransitionRows]];
Print["TCCTTransitionExact=", t5sTCCTTransitionExactCount, "/", Length[t5sHighOrderTra
Print["TCCTTransitionExactAccuracy=", t5sTCCTTransitionExactAccuracy];
Print["NeuralTransitionExact=", t5sNeuralTransitionExactCount, "/", Length[t5sHighOrde
Print["NeuralTransitionExactAccuracy=", t5sNeuralTransitionExactAccuracy];
Print["TCCTTransitionProbeAccuracy=", t5sTCCTTransitionProbeAccuracy];
Print["NeuralTransitionProbeAccuracy=", t5sNeuralTransitionProbeAccuracy];
Print["NeuralTransitionZeroAccuracy=", t5sNeuralTransitionZeroAccuracy];
Print["NeuralTransitionOneAccuracy=", t5sNeuralTransitionOneAccuracy];
Print["NeuralTransitionBalancedAccuracy=", t5sNeuralTransitionBalancedAccuracy];
Print["TCCTMinusNeuralExact=", t5sTCCTMinusNeuralExact];
Print["TCCTMinusNeuralProbe=", t5sTCCTMinusNeuralProbe];
Print["TCCTMinusNeuralTransitionExact=", t5sTCCTMinusNeuralTransitionExact];
Print["WinnerMargin=", t5sWinnerMargin];
Print["STRICT PROTOCOL PASS=", t5sStrictProtocolPass];
Print["GENERALIZATION DIAGNOSIS=", t5sGeneralizationDiagnosis];
Print["GlobalFreezeHash=", t5sGlobalFreezeHash];
Print["====="];
t5sSummary=<|
"Stage"->"S124-T5R1",
"StrictProspective"->True,
"FreshWorldSeed"->hiddenBenchmarkSeedS119B,
"SharedPerception"->True,
"PreWorldProtocolHash"->t5sPreWorldProtocolHash,
"PerceptionValidationAccuracy"->t5sPerceptionValidationAccuracy,
"TrainingMembershipPerceptionAccuracy"->t5sTrainingPerceptionAccuracy,
"MaximumTrainingInteractionOrder"->t5sMaximumTrainingInteractionOrder,
"TrueJointStates"->trueJointStateCountS119B,
"HighOrderHoldoutStates"->Length[t5sHighOrderRows],
"HighOrderTouchedBeforeFreeze"->highOrderTouchedBeforeFreezeS119B,
"InferredFactors"->inferredFactorCountS119B,
"LocalStateCounts"->learnedLocalStateCountsS119B,
"LearnedInteractionEdges"->learnedInteractionEdgesS119B,
"ConditionalTransitionCells"->Length[conditionalTransitionsS119B],
"MembershipQueriesBeforeFreeze"->membershipQueriesBeforeFreezeS119B,
"ReasonValidationAccuracy"->t5sReasonValidationAccuracy,
"ReasonValidationBalancedAccuracy"->t5sReasonValidationBalancedAccuracy,
"FinalReasonTrainingAccuracy"->t5sReasonTrainingAccuracy,
"FinalReasonTrainingBalancedAccuracy"->t5sReasonTrainingBalancedAccuracy,
"TCCTHighOrderExactAccuracy"->t5sTCCTExactAccuracy,
"NeuralHighOrderExactAccuracy"->t5sNeuralExactAccuracy,
"TCCTProbeAccuracy"->t5sTCCTProbeAccuracy,
"TCCTProbeBalancedAccuracy"->t5sTCCTProbeBalancedAccuracy,
"NeuralProbeAccuracy"->t5sNeuralProbeAccuracy,
"NeuralZeroAccuracy"->t5sNeuralProbeZeroAccuracy,
"NeuralOneAccuracy"->t5sNeuralProbeOneAccuracy,
"NeuralBalancedAccuracy"->t5sNeuralProbeBalancedAccuracy,
"HighOrderTransitionCases"->Length[t5sHighOrderTransitionRows],
"TCCTTransitionExactAccuracy"->t5sTCCTTransitionExactAccuracy,
"NeuralTransitionExactAccuracy"->t5sNeuralTransitionExactAccuracy,
"TCCTTransitionProbeAccuracy"->t5sTCCTTransitionProbeAccuracy,
"NeuralTransitionProbeAccuracy"->t5sNeuralTransitionProbeAccuracy,
"NeuralTransitionZeroAccuracy"->t5sNeuralTransitionZeroAccuracy,
"NeuralTransitionOneAccuracy"->t5sNeuralTransitionOneAccuracy,
"NeuralTransitionBalancedAccuracy"->t5sNeuralTransitionBalancedAccuracy,
"TCCTMinusNeuralExact"->t5sTCCTMinusNeuralExact,

```

```
"TCCTMinusNeuralProbe" -> t5sTCCTMinusNeuralProbe,  
"TCCTMinusNeuralTransitionExact" -> t5sTCCTMinusNeuralTransitionExact,  
"WinnerMargin" -> t5sWinnerMargin,  
"StrictProtocolPass" -> t5sStrictProtocolPass,  
"GeneralizationDiagnosis" -> t5sGeneralizationDiagnosis,  
"TCCTFreezeHash" -> frozenModelHashS119B,  
"GlobalFreezeHash" -> t5sGlobalFreezeHash,  
"HighOrderOutputsOpenedAfterAllModelsFrozen" -> True  
|>;  
t5sSummaryFile=  
FileNameJoin[  
{t5sOutputDirectory, "S124_T5R1_strict_summary.wl"}  
];  
Put[t5sSummary, t5sSummaryFile];  
Print["SummaryFile=", t5sSummaryFile];  
Print["====="];  
Print["S124-T5R1 COMPLETE"];  
Print["====="];
```

```

=====
S124-T5R1 STRICT FRESH-WORLD
S119B NEURALIZED GENERALIZATION ATTRIBUTION
PAIRWISE TRAINING -> UNSEEN HIGH-ORDER COMPOSITIONS
=====
WolframVersion=15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)
Date=Wed 19 Aug 2026 22:36:50
PublicActions={0, 1, 2, 3, 4, 5, 6, 7}
PublicProbes={0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13}
MaximumTrainingInteractionOrder=2
FreshHiddenWorldSeed=1245501
PreWorldProtocolHash=729707e01af54003aec9650fe3dd4813ea937553618aa1d5e2af5296\

> 8d328a91
HIDDEN WORLD DOES NOT EXIST YET.
=====
PHASE 1: TRAIN AND FREEZE SHARED PERCEPTION
HIDDEN WORLD STILL DOES NOT EXIST
=====
Starting training.
Optimization Method: ADAM
    Beta1: 9.00*^-1
    Beta2: 9.99*^-1
    Epsilon: 1.00*^-5
    Gradient Clipping: --
    L2 Regularization: --
    Learning Rate: 3.00*^-4
    Learning Rate Schedule: --
    Weight Clipping: --

Device: CPU
Batch Size: 64
Batches Per Round: 38
Training Examples: 2400
  % round batch  examples      inputs    learning      time      time  \

>  current      round      test    current      round      test    current\

>      round      test
    /20    /38 processed    /second      rate    elapsed      left    \

>  loss      loss      loss      acc.      acc.      acc.      error  \

>      error      error
    8      2    19      3648      2071    7.08*^-5      2s      26s  \

>  5.09*^-2    7.55*^-2    6.77*^-4      98.27%    97.41%      100%    \

>  1.73%      2.59%      0%
    16      4      6      7680      1807    1.01*^-4      4s      22s  \

>  1.09*^-3    1.03*^-3    2.51*^-4      100%      100%      100%    \

>  0%      0%      0%
    25      5    35      11968      2289    1.24*^-4      6s      19s  \

>  6.65*^-4    7.57*^-4    1.85*^-4      100%      100%      100%    \

```

>	0%	0%	0%					
33	7	22	16000	2136	1.41×10^{-4}	8s	16s	\
>	4.46×10^{-4}	4.67×10^{-4}	1.13×10^{-4}		100%	100%	100%	\
>	0%	0%	0%					
41	9	7	19904	1801	1.55×10^{-4}	10s	15s	\
>	3.37×10^{-4}	3.29×10^{-4}	7.61×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
49	10	34	24064	2188	1.68×10^{-4}	12s	12s	\
>	2.57×10^{-4}	2.73×10^{-4}	6.45×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
57	12	19	27968	2098	1.79×10^{-4}	14s	10s	\
>	2.06×10^{-4}	2.08×10^{-4}	4.82×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
66	14	4	31872	1675	1.88×10^{-4}	16s	8s	\
>	1.69×10^{-4}	1.68×10^{-4}	3.79×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
74	15	31	36032	2206	1.97×10^{-4}	18s	6s	\
>	1.35×10^{-4}	1.45×10^{-4}	3.41×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
82	17	19	40128	2120	2.05×10^{-4}	20s	4s	\
>	1.21×10^{-4}	1.18×10^{-4}	2.82×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
90	19	3	43968	1603	2.12×10^{-4}	22s	2s	\
>	1.04×10^{-4}	1.02×10^{-4}	2.38×10^{-5}		100%	100%	100%	\
>	0%	0%	0%					
99	20	30	48128	2189	2.18×10^{-4}	24s	0s	\
>	9.06×10^{-5}	9.30×10^{-5}	2.20×10^{-5}		100%	100%	100%	\

> 0% 0% 0%
 PerceptionValidationAccuracy=1.
 SHARED PERCEPTION FROZEN

=====

PHASE 2: GENERATE FRESH S119B HIDDEN WORLD

=====

FreshFactorSizesEvaluatorOnly={2, 3, 5, 4}
 TrueJointStates=120
 HighOrderHoldoutStates=74
 HIGH-ORDER OUTPUTS REMAIN SEALED

```

=====
PHASE 3: FACTOR SUPPORT DISCOVERY
HIGH-ORDER OUTPUTS SEALED
=====
InferredFactors=4
ProbeGroupSizes={4, 2, 5, 3}
ActionGroupSizes={2, 2, 2, 2}
LocalStateCounts={4, 2, 5, 3}
StoredLocalStates=14
ProductCapacity=120
=====
PHASE 4: ACTIVE PAIRWISE INTERACTION DISCOVERY
HIGH-ORDER OUTPUTS SEALED
=====
LearnedParentsByAction=<|0 -> {}, 1 -> {}, 2 -> {}, 3 -> {}, 4 -> {},

> 5 -> {2}, 6 -> {4}, 7 -> {}|>
LearnedInteractionEdges={2 -> 1, 4 -> 3}
ConditionalTransitionCells=42
TrainingMembershipPerceptionAccuracy=1.
MaximumTrainingInteractionOrder=2
MembershipQueriesBeforeFreeze=1822
HighOrderTouchedBeforeFreeze=0
TCCT MODEL FROZEN
TCCTFreezeHash=ad50bf7a8c5aadbc57ca93ae961624cb14a3793b312dccb7fcd7e7074ab080\

> ef
=====
PHASE 5: MATCHED NEURAL REASONER
USES SAME S119B MEMBERSHIP DATA
HIGH-ORDER OUTPUTS SEALED
=====
ReasonInputDimensions={17, 33}
ReasonMaxSequenceLength=16
ReasonTrainingRows=1420
ReasonValidationRows=402
Starting training.
Optimization Method: ADAM
    Beta1: 9.00*^-1
    Beta2: 9.99*^-1
    Epsilon: 1.00*^-5
    Gradient Clipping: --
    L2 Regularization: --
    Learning Rate: 3.00*^-4
    Learning Rate Schedule: --
    Weight Clipping: --

Device: CPU
Batch Size: 64
Batches Per Round: 23
Training Examples: 1420
  % round batch  examples      inputs    learning      time      time  \

>  current      round      test    current      round      test    current\

>      round      test
    /35    /23 processed /second      rate  elapsed      left  \

```

>	loss	loss	loss	acc.	acc.	acc.	error \
>	error	error					
4	2	8	1984	1065	5.45×10^{-5}	2s	52s \
>	6.51×10^{-1}	6.77×10^{-1}	5.73×10^{-1}	69.6%	68.1%	72.1%	\
>	30.4%	31.9%	27.9%				
8	3	20	4224	1202	7.59×10^{-5}	4s	45s \
>	5.76×10^{-1}	5.86×10^{-1}	5.52×10^{-1}	72.8%	73.0%	72.1%	\
>	27.2%	27.0%	27.9%				
12	5	7	6336	1095	9.21×10^{-5}	6s	43s \
>	5.54×10^{-1}	5.57×10^{-1}	4.94×10^{-1}	73.6%	73.5%	77.1%	\
>	26.4%	26.5%	22.9%				
17	6	22	8768	1252	1.07×10^{-4}	8s	40s \
>	5.23×10^{-1}	5.21×10^{-1}	4.94×10^{-1}	75.6%	74.9%	75.6%	\
>	24.4%	25.1%	24.4%				
21	8	11	11008	1152	1.19×10^{-4}	10s	37s \
>	5.09×10^{-1}	5.15×10^{-1}	4.87×10^{-1}	76.3%	76.2%	76.6%	\
>	23.7%	23.8%	23.4%				
26	9	23	13248	1257	1.30×10^{-4}	12s	35s \
>	5.25×10^{-1}	5.18×10^{-1}	4.83×10^{-1}	74.9%	75.1%	78.9%	\
>	25.1%	24.9%	21.1%				
30	11	14	15616	1197	1.40×10^{-4}	14s	33s \
>	5.06×10^{-1}	5.09×10^{-1}	4.82×10^{-1}	77.3%	77.3%	78.9%	\
>	22.7%	22.7%	21.1%				
35	13	2	17792	979	1.48×10^{-4}	16s	31s \
>	5.13×10^{-1}	5.02×10^{-1}	4.79×10^{-1}	76.6%	77.1%	80.1%	\
>	23.4%	22.9%	19.9%				
39	14	14	20032	1095	1.56×10^{-4}	18s	29s \
>	4.93×10^{-1}	5.00×10^{-1}	4.76×10^{-1}	77.7%	77.2%	80.6%	\
>	22.3%	22.8%	19.4%				
43	15	23	22080	1176	1.62×10^{-4}	20s	27s \
>	4.96×10^{-1}	4.97×10^{-1}	4.80×10^{-1}	77.7%	77.5%	78.9%	\
>	22.3%	22.5%	21.1%				
48	17	15	24512	1187	1.69×10^{-4}	22s	25s \

>	4.90* ⁻¹	4.95* ⁻¹	4.82* ⁻¹	77.5%	77.2%	78.4%	\
>	22.5%	22.8%	21.6%				
	52	19	4	26752	1047	1.75* ⁻⁴	24s 23s \
>	4.74* ⁻¹	4.77* ⁻¹	4.66* ⁻¹	78.2%	77.7%	80.8%	\
>	21.8%	22.3%	19.2%				
	56	20	16	28992	1118	1.81* ⁻⁴	26s 20s \
>	4.71* ⁻¹	4.77* ⁻¹	4.63* ⁻¹	78.8%	78.3%	80.6%	\
>	21.2%	21.7%	19.4%				
	60	22	3	31104	994	1.86* ⁻⁴	28s 19s \
>	4.80* ⁻¹	4.71* ⁻¹	4.68* ⁻¹	78.5%	78.3%	78.6%	\
>	21.5%	21.7%	21.4%				
	65	23	14	33280	1086	1.91* ⁻⁴	30s 17s \
>	4.76* ⁻¹	4.72* ⁻¹	4.70* ⁻¹	78.4%	79.1%	79.9%	\
>	21.6%	20.9%	20.1%				
	69	24	23	35328	1157	1.95* ⁻⁴	32s 15s \
>	4.61* ⁻¹	4.66* ⁻¹	4.52* ⁻¹	79.8%	79.2%	81.3%	\
>	20.2%	20.8%	18.7%				
	73	26	13	37632	1098	2.00* ⁻⁴	34s 13s \
>	4.52* ⁻¹	4.47* ⁻¹	4.57* ⁻¹	79.4%	79.7%	79.9%	\
>	20.6%	20.3%	20.1%				
	77	27	23	39744	1183	2.04* ⁻⁴	37s 11s \
>	4.46* ⁻¹	4.47* ⁻¹	4.55* ⁻¹	80.2%	80.0%	80.1%	\
>	19.8%	20.0%	19.9%				
	82	29	14	42112	1172	2.08* ⁻⁴	39s 9s \
>	4.40* ⁻¹	4.50* ⁻¹	4.53* ⁻¹	81.3%	80.8%	80.1%	\
>	18.7%	19.2%	19.9%				
	86	31	2	44288	959	2.12* ⁻⁴	41s 7s \
>	4.40* ⁻¹	4.29* ⁻¹	4.43* ⁻¹	80.3%	80.8%	80.8%	\
>	19.7%	19.2%	19.2%				
	90	32	14	46528	1122	2.16* ⁻⁴	43s 5s \
>	4.17* ⁻¹	4.18* ⁻¹	4.46* ⁻¹	81.8%	81.8%	80.8%	\
>	18.2%	18.2%	19.2%				
	94	33	23	48576	1149	2.19* ⁻⁴	45s 3s \
>	4.31* ⁻¹	4.29* ⁻¹	4.30* ⁻¹	81.0%	81.0%	81.8%	\

```

> 19.0%    19.0%    18.2%
  99      35     12    50816      1091  2.22*^-4      47s      1s  \

> 4.27*^-1 4.23*^-1 4.42*^-1      81.8%    81.9%    80.8%    \

> 18.2%    18.1%    19.2%
ReasonValidationAccuracy=0.820896
ReasonValidationZeroAccuracy=0.958621
ReasonValidationOneAccuracy=0.464286
ReasonValidationBalancedAccuracy=0.711453
Starting training.
Optimization Method: ADAM
      Beta1: 9.00*^-1
      Beta2: 9.99*^-1
      Epsilon: 1.00*^-5
      Gradient Clipping: --
      L2 Regularization: --
      Learning Rate: 3.00*^-4
      Learning Rate Schedule: --
      Weight Clipping: --

Device: CPU
Batch Size: 64
Batches Per Round: 29
Training Examples: 1822
  % round batch  examples    inputs    learning    time    time  \

> current      round    current      round    current      round
   /35      /29 processed   /second      rate  elapsed      left  \

> loss      loss      acc.      acc.      error      error
  4      2      8      2368      1256  5.84*^-5      2s      56s  \

> 6.46*^-1 6.59*^-1      68.6%    68.0%    31.4%    32.0%
  7      3     18      4864      1260  8.12*^-5      4s      50s  \

> 5.59*^-1 5.74*^-1      73.8%    72.6%    26.2%    27.4%
 11      4     28      7360      1221  9.89*^-5      6s      48s  \

> 5.35*^-1 5.52*^-1      75.3%    74.2%    24.7%    25.8%
 15      6      9      9856      1257  1.13*^-4      8s      45s  \

> 5.10*^-1 5.14*^-1      76.4%    76.5%    23.6%    23.5%
 19      7     18     12288      1180  1.25*^-4     10s      44s  \

> 5.25*^-1 5.15*^-1      76.2%    76.5%    23.8%    23.5%
 23      8     26     14656      1166  1.36*^-4     12s      42s  \

> 4.92*^-1 5.13*^-1      78.3%    76.9%    21.7%    23.1%
 26     10      6     17088      1215  1.45*^-4     14s      40s  \

> 5.06*^-1 5.06*^-1      77.8%    77.8%    22.2%    22.2%
 30     11     14     19456      1192  1.54*^-4     16s      38s  \

> 5.08*^-1 5.01*^-1      76.9%    77.9%    23.1%    22.1%
 34     12     23     21888      1199  1.61*^-4     18s      36s  \

```

>	4.97 ⁻¹	4.97 ⁻¹	77.5%	77.5%	22.5%	22.5%	
37	14	3	24320	1205	1.69 ⁻⁴	20s	34s \
>	4.86 ⁻¹	4.87 ⁻¹	78.3%	78.2%	21.7%	21.8%	
41	15	12	26752	1216	1.75 ⁻⁴	22s	32s \
>	4.76 ⁻¹	4.86 ⁻¹	78.9%	78.1%	21.1%	21.9%	
45	16	23	29312	1254	1.82 ⁻⁴	24s	30s \
>	4.86 ⁻¹	4.83 ⁻¹	78.3%	78.8%	21.7%	21.2%	
49	18	3	31744	1201	1.88 ⁻⁴	26s	28s \
>	4.78 ⁻¹	4.77 ⁻¹	78.9%	79.0%	21.1%	21.0%	
53	19	12	34176	1210	1.93 ⁻⁴	28s	26s \
>	4.62 ⁻¹	4.67 ⁻¹	79.6%	79.0%	20.4%	21.0%	
56	20	21	36608	1242	1.98 ⁻⁴	30s	24s \
>	4.70 ⁻¹	4.68 ⁻¹	79.7%	79.7%	20.3%	20.3%	
60	22	2	39104	1240	2.03 ⁻⁴	32s	21s \
>	4.71 ⁻¹	4.69 ⁻¹	79.0%	78.8%	21.0%	21.2%	
64	23	11	41536	1241	2.07 ⁻⁴	34s	19s \
>	4.66 ⁻¹	4.63 ⁻¹	79.6%	79.9%	20.4%	20.1%	
68	24	21	44032	1225	2.12 ⁻⁴	36s	17s \
>	4.56 ⁻¹	4.67 ⁻¹	79.9%	79.2%	20.1%	20.8%	
72	26	2	46528	1255	2.16 ⁻⁴	39s	15s \
>	4.62 ⁻¹	4.56 ⁻¹	79.4%	79.6%	20.6%	20.4%	
75	27	10	48896	1167	2.19 ⁻⁴	41s	13s \
>	4.51 ⁻¹	4.50 ⁻¹	80.3%	80.2%	19.7%	19.8%	
79	28	19	51328	1209	2.23 ⁻⁴	43s	11s \
>	4.47 ⁻¹	4.51 ⁻¹	80.2%	80.3%	19.8%	19.7%	
83	29	28	53760	1205	2.26 ⁻⁴	45s	9s \
>	4.48 ⁻¹	4.53 ⁻¹	80.9%	80.3%	19.1%	19.7%	
86	31	7	56128	1200	2.29 ⁻⁴	47s	7s \
>	4.54 ⁻¹	4.49 ⁻¹	79.8%	80.2%	20.2%	19.8%	
90	32	15	58496	1177	2.32 ⁻⁴	49s	5s \
>	4.30 ⁻¹	4.42 ⁻¹	81.2%	80.5%	18.8%	19.5%	
94	33	23	60864	1179	2.35 ⁻⁴	51s	3s \
>	4.30 ⁻¹	4.27 ⁻¹	80.5%	81.1%	19.5%	18.9%	
98	35	4	63360	1233	2.38 ⁻⁴	53s	1s \

> 4.11⁻¹ 4.25⁻¹ 82.0% 81.5% 18.0% 18.5%
 FinalReasonTrainingAccuracy=0.830955
 FinalReasonTrainingBalancedAccuracy=0.703291

=====

GLOBAL FREEZE COMPLETE
SharedPerceptionFrozen=True
TCCTFrozen=True
NeuralReasonerFrozen=True
HighOrderOutputsOpened=False
ReasonMaxSequenceLength=16
GlobalFreezeHash=06269f1e35c0831451cb1056d0dbd4c51f303574fa330fd2af295973044f\

> 30e9

=====

PHASE 6: FIRST OPENING OF FRESH HIGH-ORDER HOLDOUT
NO MODEL CHANGES AFTER THIS LINE

=====

Opening high-order transition audit...

=====

S124-T5R1 FINAL STRICT FRESH-WORLD SUMMARY

=====

FreshWorldSeed=1245501
StrictProspective=True
SharedPerception=True
PreWorldProtocolHash=729707e01af54003aec9650fe3dd4813ea937553618aa1d5e2af5296\

> 8d328a91

PerceptionValidationAccuracy=1.
TrainingMembershipPerceptionAccuracy=1.
MaximumTrainingInteractionOrder=2
TrueJointStates=120
HighOrderHoldoutStates=74
HighOrderTouchedBeforeFreeze=0
InferredFactors=4
LocalStateCounts={4, 2, 5, 3}
LearnedInteractionEdges={2 -> 1, 4 -> 3}
ConditionalTransitionCells=42
MembershipQueriesBeforeFreeze=1822
ReasonValidationAccuracy=0.820896
ReasonValidationBalancedAccuracy=0.711453
FinalReasonTrainingAccuracy=0.830955
FinalReasonTrainingBalancedAccuracy=0.703291
TCCTHighOrderExact=74/74
TCCTHighOrderExactAccuracy=1.
NeuralHighOrderExact=0/74
NeuralHighOrderExactAccuracy=0.
TCCTProbeAccuracy=1.
TCCTProbeBalancedAccuracy=1.
NeuralProbeAccuracy=0.535714
NeuralZeroAccuracy=0.682432
NeuralOneAccuracy=0.168919
NeuralBalancedAccuracy=0.425676
HighOrderTransitionCases=592
TCCTTransitionExact=592/592
TCCTTransitionExactAccuracy=1.
NeuralTransitionExact=0/592
NeuralTransitionExactAccuracy=0.
TCCTTransitionProbeAccuracy=1.

```
NeuralTransitionProbeAccuracy=0.560569
NeuralTransitionZeroAccuracy=0.693919
NeuralTransitionOneAccuracy=0.227196
NeuralTransitionBalancedAccuracy=0.460557
TCCTMinusNeuralExact=1.
TCCTMinusNeuralProbe=0.464286
TCCTMinusNeuralTransitionExact=1.
WinnerMargin=0.02
STRICT PROTOCOL PASS=True
GENERALIZATION DIAGNOSIS=STRICT_FRESH_WORLD_TCCT_HIGH_ORDER_GENERALIZATION_AD\
```

```
> VANTAGE
GlobalFreezeHash=06269f1e35c0831451cb1056d0dbd4c51f303574fa330fd2af295973044f\
```

```
> 30e9
=====
SummaryFile=E:\TCCT_CODEX_HANDOFF_2026-08-13\S97A_ReadoutBaseline_Development\
```

```
> \S124_T5R1_Output\S124_T5R1_strict_summary.wl
=====
S124-T5R1 COMPLETE
=====
```

In []: